



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Department of Computer Science
Faculty of Computing

Technical Report (WorkBeacon)

By MemeMaker

Programme	: Bachelor of Computer Science
Subject Code	: SECJ3623
Subject Name	: PENGATURCARAAN APLIKASI MOBIL
Session-Sem	: 2025/2026-01

Prepared by : 1) MUHAMAD ARIF BIN JOHAR (SX211702ECRHF04)
2) MOHAMED FIRDAUS BIN ZAIDI (SX200604ECJHS04)
3) MUHAMMAD HIZWAN BIN ZAMERI
(SX211682ECJHF03)
4) MUHAMMAD HAZIM BIN ADNAN (SX200750ECJHS04)

Section : 01

Lecturer : DR. MOHD SHAHIZAN BIN OTHMAN

Table of Contents

1. Introduction.....	3
2. Development Approach and Workflow	3
2.1 Development Methodology	3
2.2 Development Workflow	4
3. Software, Tools, and Frameworks.....	6
3.1 Development Environment.....	6
3.2 Programming Languages and Frameworks	7
3.3 Supporting Tools	8
4. Database and Data Handling Design.....	10
4.1 Data Storage Architecture.....	10
4.2 Data Models	12
4.3 Data Flow	15
5. CRUD Operations.....	18
5.1 Create Operations	18
5.2 Read Operations	19
5.3 Update Operations	19
5.4 Delete Operations	20
6. Important Source Code Snippets.....	21
6.1 Authentication and Role Handling	21
6.2 Alert Broadcasting Logic	22
6.3 Incident Reporting Mechanism.....	22
6.4 Data Retrieval and Update Operations	23
7. Technical Challenges and Solutions.....	25
7.1 Identified Challenges.....	25
7.2 Implemented Solutions.....	26
8. Conclusion.....	29

1. Introduction

WorkBeacon is a mobile application developed for workplace safety, incident reporting, and staff visibility. Staff can report incidents (with categories, descriptions, locations, and photos) and view alerts sent by administrators, while administrators can manage incidents, broadcast alerts, browse a staff directory, and view staff locations on a map. The project was developed by the MemeMaker team for the course SECJ3623 Pengaturcaraan Aplikasi Mobil (Mobile Application Programming) 2025/2026-01.

This technical report documents the design and implementation of WorkBeacon. Section 2 describes the development approach and workflow, including the methodology and processes used throughout the project. Section 3 presents the software, tools, and frameworks: the development environment, programming languages and frameworks, and supporting tools. Section 4 explains the database and data-handling design, covering data storage architecture, data models, and data flow. Section 5 details all CRUD (Create, Read, Update, Delete) operations, organised by operation type (Create, Read, Update, Delete), with their locations in the application and justifications. Section 6 highlights important source code snippets that illustrate key implementation choices. Section 7 discusses technical challenges encountered and the solutions adopted. Section 8 concludes the report with a summary and reflections on the project.

2. Development Approach and Workflow

2.1 Development Methodology

The WorkBeacon project followed Scrum. Development was organised in time-boxed sprints (e.g. one to two weeks). Each sprint had a clear goal (e.g. ‘Incident reporting end-to-end’) and produced a potentially shippable increment. The team of four held sprint planning, daily stand-ups, and sprint reviews to align progress and adapt the backlog.. This choice was made to suit a small team (four developers), a fixed semester timeline, and a product with distinct feature areas (staff vs. admin) that could be developed and tested incrementally.

Core principles:

- **Iterative development** — The app was built in short cycles. Each cycle focused on a subset of features (e.g. authentication and profiles, then incident reporting, then alerts, then admin tools). This allowed the team to integrate and test frequently and to adjust plans when needed.
- **Feature-oriented work** — Requirements were grouped by user role and feature (e.g. “Staff: report incident”, “Admin: send alerts”). Tasks were assigned so that different parts of the app could progress in parallel where possible, matching the split between staff and admin screens in the codebase.
- **Incremental delivery** — A minimal working version (e.g. login, basic profiles, one core flow) was targeted first. New capabilities (incident reporting with photos, alerts, staff directory, live map) were added in subsequent iterations so that the application was always in a testable state.
- **Continuous integration and testing** — Code was integrated into a shared codebase (e.g. via version control). Builds were run regularly to catch integration issues early. Testing was performed at the end of each iteration and when major features were completed.

Application to WorkBeacon:

The methodology was applied by aligning iterations with major functional areas: (1) project setup, authentication, and user profiles; (2) staff incident reporting and listing; (3) alerts creation and viewing; (4) admin incident management and status updates; (5) staff directory and live locations (map). This order ensured that core data (profiles, incidents, alerts) and CRUD operations were stable before building admin and map features that depend on them. This approach helped the team manage scope, deliver a working product within the course timeline, and document CRUD operations and technical decisions in a structured way for this report.

2.2 Development Workflow

The overall workflow followed a structured sequence from requirements to deployment, with task tracking and progress monitoring throughout.

Requirement analysis and feature identification

The team analysed the problem domain (workplace safety, incident reporting, staff visibility) and identified functional requirements by user role. Staff requirements included authentication, profile management, incident reporting (with categories, description, location, and photo), viewing incidents and alerts, and alert history. Admin requirements included a central dashboard, viewing and managing all incidents (including status updates), sending alerts (with optional images and audience targeting), a staff directory, and live staff locations on a map. Non-functional needs such as real-time updates (Firestore streams), secure access (Firebase Auth), and file storage (incident and profile images) were also captured. This phase produced a clear feature list and scope for the application.

UI/UX design and prototyping

Screens and user flows were designed for both staff and admin. Staff flows covered login, dashboard, report incident, incidents list and details, alert history and details, and profile view and edit. Admin flows covered dashboard, all incidents (list and detail with status update), send alert, staff directory, and live locations map. Layouts, navigation, and key interactions were agreed (and where applicable, prototyped) so that development could proceed with a shared understanding of the user interface and experience.

Application development and integration

Development was carried out in line with the chosen methodology (Section 2.1), building features incrementally. The codebase was organised into modules such as authentication (login, sign-up), staff screens (dashboard, incidents, report incident, alerts, profile), admin screens (dashboard, incidents, alerts, directory, live map), shared services (e.g. profile service), and reusable widgets (e.g. incident card). Firebase (Authentication, Firestore, Storage) was integrated early and used consistently for CRUD operations and real-time data. Work was merged into a shared codebase via version control to maintain a single, integrable application throughout the cycle.

Testing and refinement

As features were completed, they were tested manually: form validation, Firestore and Storage rules, navigation, and role-based behaviour (staff vs admin). Issues and edge cases were logged, prioritised, and addressed in subsequent work. Refinements included fixing validation, improving error handling and user feedback (e.g. SnackBars), and ensuring that

create, read, update, and delete operations behaved correctly for staff profiles, incidents, and alerts. This iterative testing and refinement continued until the application met the agreed requirements and quality bar.

Final deployment preparation

In the final phase, the team prepared for deployment: verifying Firebase configuration (Auth, Firestore, Storage rules and indexes), ensuring environment setup (e.g. `.env` for optional Mapbox) and build instructions were documented in the README, and producing release builds (e.g. Android APK/App Bundle and iOS archive). Documentation, including this technical report, was completed to capture design, CRUD operations, and development decisions.

Task tracking and progress monitoring

Task tracking and progress monitoring were conducted throughout the development cycle to ensure timely delivery and quality control. The team used **GitHub Project** and the **GitHub repository** to manage tasks, track progress, and coordinate work. Backlog items and feature tasks were created and assigned; status was updated as work moved from planning to in progress to done. The repository was used for version control, code review, and integration, so that the workflow from requirement analysis through to deployment preparation was visible and manageable for the whole team.

3. Software, Tools, and Frameworks

3.1 Development Environment

The development environment for WorkBeacon was set up to support Flutter application development, Firebase integration, and builds for multiple target platforms.

Host system and toolchain

Development was carried out on host machines running a supported operating system (e.g. Windows, macOS, or Linux). The Flutter SDK (stable channel) was installed and added to the system path so that the `flutter` and `dart` commands were available from the terminal. The project constrains the Dart SDK to version **3.9.2** or compatible (defined in `pubspec.yaml` under `environment.sdk`). Git was used for version control, with the repository hosted on GitHub to support collaboration, task tracking, and code integration as described in Section 2.2.

Integrated development environment

Team members used an IDE or editor suitable for Flutter development, such as Android Studio with the Flutter and Dart plugins, Visual Studio Code with the Flutter extension, or Cursor. These environments provide Dart/Flutter syntax support, running and debugging on devices or emulators, and integration with the Flutter CLI for commands such as `flutter run`, `flutter pub get`, and `flutter build`.

Android development setup

For Android builds and debugging, the Android SDK was installed (typically via Android Studio or standalone SDK). The project is configured for **Android** using Kotlin and Gradle, with **Java 11** for source and target compatibility. The Firebase Android configuration file (`google-services.json`) was placed in `android/app/` and the Google Services Gradle plugin was applied so that Firebase (Authentication, Firestore, Storage) could be used in the app. The application identifier is `com.example.workbeacon`.

iOS development setup (where applicable)

For building and running on iOS, development was performed on macOS with Xcode installed. The iOS app was registered in the Firebase Console, and the generated `GoogleService-Info.plist` was added to the iOS project. The Flutter project supports the **ios** platform so that the app can be run on the iOS Simulator or on physical iOS devices and prepared for archive and distribution via Xcode.

Firebase and optional configuration

A Firebase project was created in the Firebase Console, with Authentication (e.g. Email/Password), Cloud Firestore, and Firebase Storage enabled. Android and iOS apps were registered and the respective configuration files were added to the repository as described above. Optional map tiles (Mapbox) were configured via a `.env` file (copied from `.env.example`) containing `MAPBOX_ACCESS_TOKEN`; the app runs without this file, using default map tiles when no token is set.

Target platforms and testing

The project is configured for multiple target platforms: **Android**, **iOS**, **web**, **Windows**, **macOS**, and **Linux**. Primary development and testing were focused on mobile (Android and, where available, iOS), using either device emulators/simulators or physical devices connected via USB or wireless debugging. This environment allowed the team to develop, run, and test WorkBeacon consistently and to produce release builds (e.g. Android APK/App Bundle and iOS archive) for deployment preparation.

3.2 Programming Languages and Frameworks

WorkBeacon is built with Dart as the main application language and Flutter as the UI framework, with Firebase for backend services and a small set of libraries for maps, images, and configuration.

Programming languages

- **Dart** — The application logic, UI, and integration with Firebase and plugins are written in **Dart**. The project uses the Dart SDK constrained to **^3.9.2** (see `pubspec.yaml`). Dart was chosen as the primary language because it is the language of the Flutter framework and supports `async/await`, strong typing, and a single codebase for multiple platforms.
- **Kotlin** — The Android native layer (e.g. `MainActivity` and Gradle build scripts) uses **Kotlin**. The Android module is configured with **Java 11** as the source and target compatibility level (and JVM target for Kotlin), which matches Flutter's Android toolchain requirements.
- **Platform-native languages** — For **iOS**, the Flutter-generated runner uses the standard Xcode toolchain (Swift/Objective-C). The team did not need to write custom native code; Flutter's embedding and plugins handle the platform integration.

Frameworks

- **Flutter** — The project uses the **Flutter** SDK (stable channel) as the cross-platform UI framework. Flutter provides the widget model, rendering, and access to platform APIs via plugins. The app enables **Material Design** (`uses-material-design: true`) for consistent Android-style components and also uses **Cupertino** icons for iOS-style visuals where appropriate. Flutter allows a single codebase to target Android, iOS, web, Windows, macOS, and Linux.
- **Firebase** — Backend and app services are provided by **Firebase**:
 - **firebase_core** (^3.15.2) — Initialisation and configuration.
 - **firebase_auth** (^5.7.0) — Email/password authentication and user management.
 - **cloud_firestore** (^5.0.0) — NoSQL database for staff profiles, incidents, and alerts, with real-time listeners.
 - **firebase_storage** (^12.0.0) — File storage for incident photos and profile images.

On Android, Firebase is used via the Firebase BoM (34.7.0) and the Google Services plugin; the same Firebase project is used for the iOS app via its own configuration file.

Supporting libraries

- **flutter_map** (^8.0.0) and **latlong2** (^0.9.1) — Map display and coordinates for the admin live locations screen; optional Mapbox tiles can be supplied via environment configuration.
- **image_picker** (^1.0.7) — Picking images from the device camera or gallery for incident reports and profile pictures.
- **flutter_dotenv** (^5.2.1) — Loading optional environment variables (e.g. MAPBOX_ACCESS_TOKEN) from a `.env` file without hardcoding secrets.

Development and quality

- **flutter_test** — Testing support for the Flutter app.
- **flutter_lints** (^5.0.0) — Linting rules to enforce Dart/Flutter style and catch common issues.

Together, Dart and Flutter form the core of the application, Firebase provides authentication, database, and storage, and the listed libraries extend the app with maps, image picking, and configuration—all aligned with the features and CRUD operations described elsewhere in this report.

3.3 Supporting Tools

Additional tools were used to support version control, project management, UI design, and testing activities. These tools improved collaboration, code quality, and development efficiency.

Version control

Git was used for version control so that all source code, configuration files, and assets were tracked in a single repository. The repository was hosted on **GitHub**, giving the team a central place to clone, commit, push, and pull changes; to use branches for features or fixes; and to review code before merging. GitHub also provided backup, history, and a clear record of who changed what and when, which supported collaboration and made it easier to trace issues and roll back if needed.

Project management

GitHub Project (and the same **GitHub** repository) was used for task and progress management, as described in Section 2.2. Tasks, user stories, and bugs were recorded and assigned; status was updated as work moved from backlog to in progress to done. This kept the team aligned on priorities and deadlines and gave visibility into what was completed and what was still pending, supporting timely delivery and coordination across the four team members.

UI design

UI design and layout were supported by design and prototyping tools (using **Figma**) or by agreed sketches and screen flows. These were used to define navigation, screen layouts, and key interactions for staff and admin flows before or alongside implementation. A shared design reference helped keep the built interface consistent and reduced rework during development.

Testing and code quality

Testing and quality were supported in several ways. **Flutter's built-in tooling** (e.g. **Flutter DevTools** for debugging and profiling, and the **flutter test** runner) was used during development. **Device emulators and simulators** (Android Emulator, iOS Simulator) and **physical devices** were used to run the app and verify behaviour on different screen sizes and OS versions. **flutter_lints** was used in the project to enforce Dart/Flutter style and catch common mistakes, improving code quality. Manual testing of forms, Firebase integration, and role-based behaviour (staff vs admin) was carried out as features were completed, as noted in Section 2.2. Together, these tools helped the team find and fix issues early and maintain a stable codebase.

Summary

In summary, Git and GitHub supported version control and collaboration; GitHub Project supported task and progress tracking; design tools or wireframes supported UI consistency; and Flutter tooling, emulators, linters, and manual testing supported quality and efficiency. These supporting tools complemented the development environment (Section 3.1) and the programming languages and frameworks (Section 3.2) used to build WorkBeacon.

4. Database and Data Handling Design

4.1 Data Storage Architecture

WorkBeacon uses a **cloud-first** data storage architecture. All persistent data lives in **Firebase** (Google Cloud). The Flutter app runs on the client and talks to Firebase over the internet using the official SDKs. There is no custom application server; Firebase Authentication, Cloud Firestore, and Firebase Storage provide identity, database, and file storage.

Overview

- **Firebase Authentication** holds user identities (e.g. email/password). It issues an authenticated user ID (`uid`) used everywhere else.
- **Cloud Firestore** is the main database: a NoSQL document store. Data is organised in **collections** and **documents**. The app uses three collections: `staff_profiles`, `staff_incidents`, and `alerts`. Access is enforced by **Firestore security rules** so that only authenticated users (and, where required, the profile owner or appropriate role) can read or write.
- **Firebase Storage** holds binary files: incident photos and profile images. Files are stored under paths such as `incidents/{userId}/...` and `profiles/{userId}/...`. The app stores only the resulting **download URLs** in Firestore; Storage access is enforced by **Storage security rules**.

Firestore structure

- **staff_profiles** — One document per user, with document ID = user `uid`. Fields include `uid`, `email`, `fullName`, `phone`, `department`, `position`, `staffId`, `userType`, `profileImageUrl`, and `timestamps`. Used for profile screens and the staff directory.
- **staff_incidents** — One document per incident report. Document IDs are auto-generated. Fields include `category`, `description`, `location`, `reportedBy`, `reportedByEmail`, `reportedByName`, `status`, `photoUrl`, and `timestamps`. Used for incident listing and management.
- **alerts** — One document per alert. Document IDs are auto-generated. Fields include `alertType`, `title`, `description`, `location`, `sendTo`, `requireAcknowledgment`, `imageUrl`, `createdBy`, `createdByEmail`, `createdByName`, `status`, and `timestamps`. Used for sending and viewing alerts.

Storage structure

- **profiles/{userId}/{filename}** — Profile pictures. Only the owning user can write; any authenticated user can read (e.g. for directory).
- **incidents/{userId}/{filename}** — Incident photos. Any authenticated user can read and write (staff upload when reporting; anyone with access can view).

Security model

Access is defined in `firebase.rules` and `storage.rules`.
Firestore: `staff_profiles` allow read for any authenticated user, create/update only when `request.auth.uid == userId`, and delete is disabled. `staff_incidents` and `alerts` allow read/write for authenticated users (rules do not currently restrict by role; app logic differentiates staff vs admin). Storage: profile images are restricted by `userId`; incident images require authentication only. This architecture keeps a single source of truth in the cloud, supports real-time listeners (e.g. `Firestore snapshots()`), and scales with Firebase without the team operating their own database or file servers.

Figure 4.1 Shows the diagram of the app data storage architecture

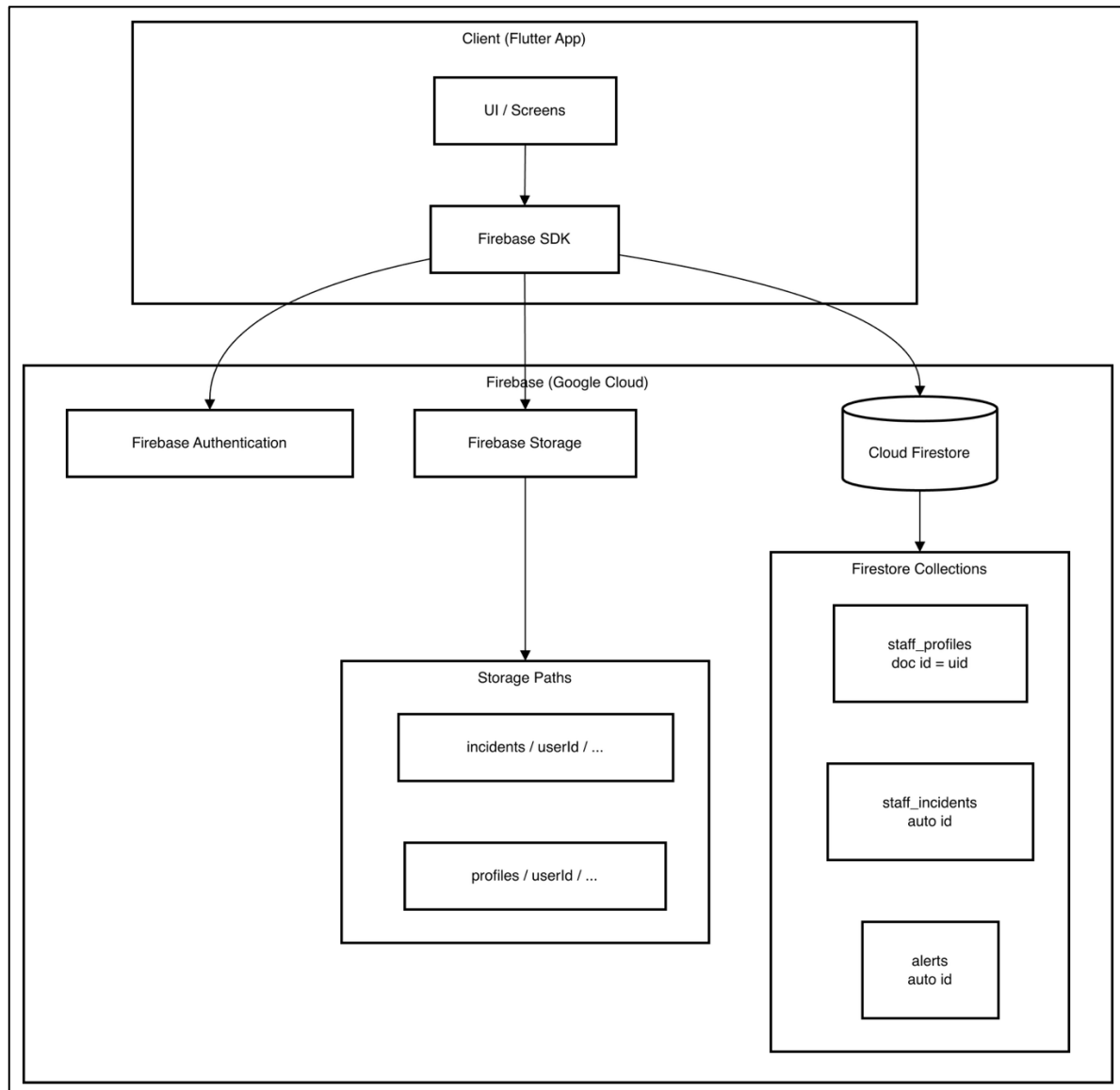


Figure 4.1 WorkBeacon Data Storage Storage Architecture

4.2 Data Models

The application uses three main Firestore collections. Each is a set of documents with a consistent set of fields. The following tables describe the structure used in the codebase. Firebase Storage is used for files; the app only stores their URLs in Firestore, so Storage is not modelled as separate “entities” with fields beyond path and URL.

Staff Profile

The `staff_profiles` collection holds one document per user. The document ID is the user’s Firebase Auth UID. The model supports profile display, edit, and the staff directory.

Table 4.2.0 Staff Collection

Field	Type	Required	Description
uid	string	Yes	Firebase Auth user ID (same as document ID).
email	string	Yes	User’s email address.
fullName	string	Yes	Display name.
phone	string	No	Phone number (may be empty).
department	string	No	Department (e.g. for directory and alerts targeting).
position	string	No	Job title/position.
staffId	string	No	Internal staff identifier.
userType	string	Yes	Role: "staff" or "admin" (used for routing).
profileImageUrl	string	No	URL of profile photo in Firebase Storage (<code>profiles/{userId}/...</code>).
joinDate	timestamp	No	Server timestamp when profile was created.
createdAt	timestamp	Yes	Server timestamp at creation.
updatedAt	timestamp	Yes	Server timestamp at last update.
status	string	No	Optional; if present, used in directory (e.g. “Active”).

Staff Incident

The `staff_incidents` collection holds one document per incident report. Document IDs are auto-generated by Firestore. Used for reporting, listing, and admin status updates.

Table 4.2.1 Staff Incident Collection

Field	Type	Required	Description
category	string	Yes	One of: Safety Hazard, Equipment Malfunction, Security Issue, Maintenance Request, Other.
description	string	Yes	Incident description.
location	string	No	Text location (may be null).
reportedBy	string	Yes	UID of the reporting user.
reportedByEmail	string	No	Reporter’s email (denormalised).
reportedByName	string	No	Reporter’s display name (denormalised).

status	string	Yes	Workflow state: "pending" (or shown as "open"), "in progress", "resolved", etc.
photoUrl	string	No	URL of incident photo in Firebase Storage (incidents/{userId}/...), or null.
createdAt	timestamp	Yes	Server timestamp at creation.
updatedAt	timestamp	Yes	Server timestamp at last update (e.g. status change).

Alert

The alerts collection holds one document per broadcast alert. Document IDs are auto-generated. Used for sending alerts (admin) and viewing them (staff and admin).

Field	Type	Required	Description
alertType	string	Yes	Type of alert (e.g. Info, Warning, Emergency).
title	string	Yes	Alert title.
description	string	Yes	Alert body.
location	string	No	Related location (may be null).
sendTo	string	Yes	Audience: "all" or a department name.
requireAcknowledgment	boolean	Yes	Whether acknowledgment is required.
imageUrl	string	No	URL of optional image in Storage, or null.
createdBy	string	Yes	UID of admin who created the alert.
createdByEmail	string	No	Creator's email (denormalised).
createdByName	string	No	Creator's display name (denormalised).
createdAt	timestamp	Yes	Server timestamp at creation.
status	string	Yes	e.g. "active" (used in queries).

Relationships Between Models

- **Staff Profile** is tied to Firebase Auth by **uid** (1:1 with Auth user). There are no foreign-key links in Firestore; the app uses **uid** and **reportedBy** / **createdBy** to relate data.
- **Staff Incident** references the reporter via **reportedBy** (UID). **photoUrl** points to a file in Storage; there is no separate "incident photo" document.
- **Alert** references the creator via **createdBy** (UID). **imageUrl** points to an optional file in Storage.

The diagram below (Figure 4.2) summarises the three document models and their main reference fields (no separate Storage entities; Storage URLs are stored in the fields above).

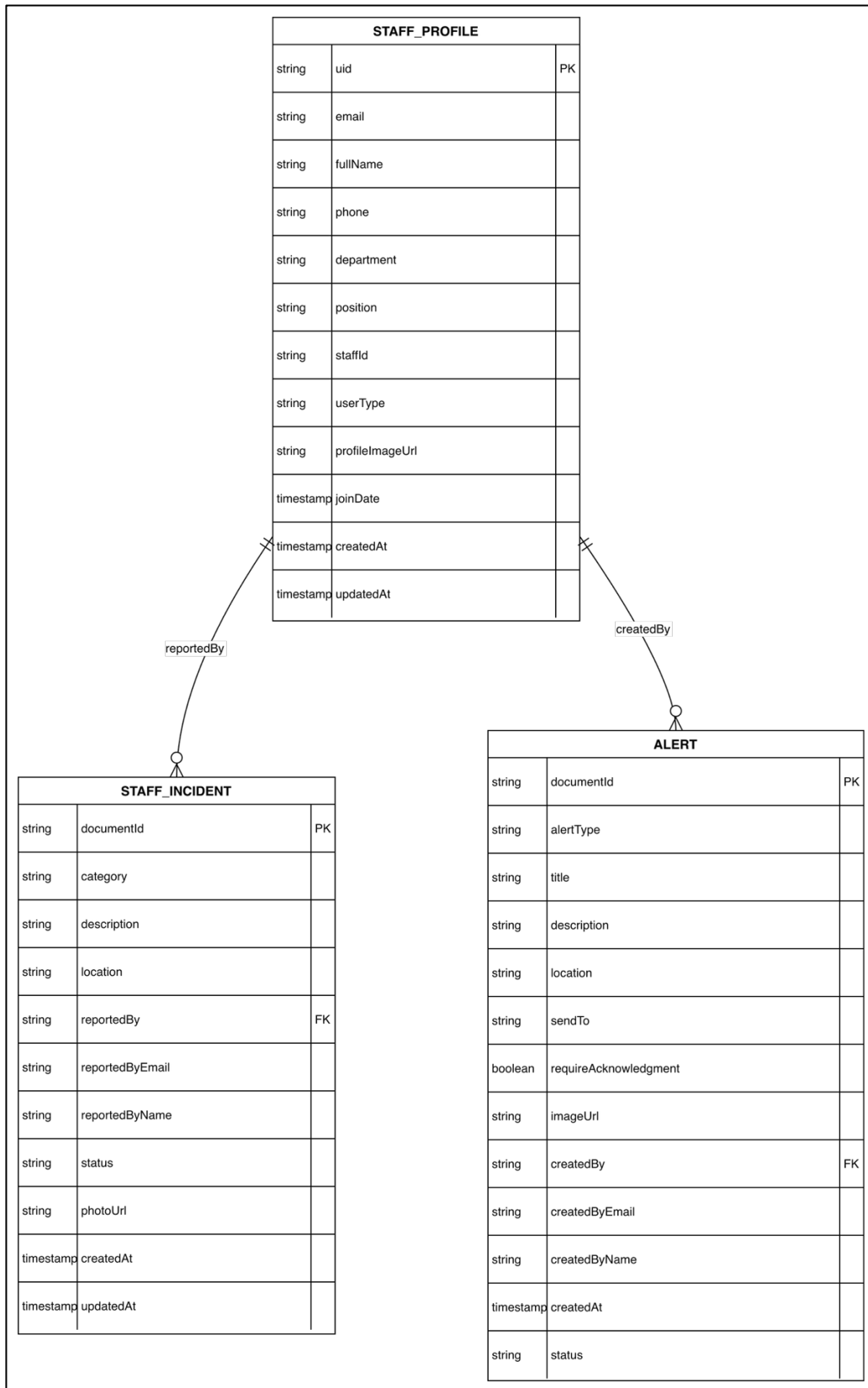


Figure 4.2 Data Model Diagram

4.3 Data Flow

This section explains how data flows between the user interface, application logic, and storage layer, and how the system keeps data consistent and intact.

Overview

Data moves in two main directions. Outbound: user actions in the UI trigger application logic (validation, formatting), which then calls the Firebase SDK to read or write Authentication, Firestore, or Storage. Inbound: Firebase (mainly Firestore) pushes updates to the app via real-time listeners; the app updates its state and the UI re-renders. There is no separate backend server; the Flutter app talks to Firebase from the client, and security is enforced by Firebase security rules and by validating input before writes.

Outbound flow (UI → application logic → storage)

1. **User action** — The user interacts with the UI (e.g. submit report, edit profile, send alert, delete incident).
2. **Validation and preparation** — The application logic checks input (e.g. required fields, format), builds the payload (maps and timestamps), and may upload files first (e.g. incident or profile image) to Firebase Storage.
3. **Storage write** — The app calls the Firebase SDK to write to Firestore (e.g. set, add, update, delete) or to Storage (putFile). For documents that reference images, the app stores the Storage download URL in the Firestore document after a successful upload.
4. **Feedback** — On success, the UI shows a message (e.g. SnackBar) and may navigate away or clear the form; on failure, it shows an error and keeps the user on the same screen.

Inbound flow (storage → application logic → UI)

1. **Subscription** — Screens that need live data attach a Firestore listener (e.g. `collection(...).snapshots()` or a query with `where/orderBy`). Authentication state is observed via Firebase Auth (e.g. `authStateChanges`).
2. **Data delivery** — Firestore sends initial data and then incremental updates whenever the collection or query result changes. The app maps snapshot documents to in-memory data (e.g. lists of incidents or alerts).
3. **UI update** — Widgets that depend on this data (e.g. `StreamBuilder`) rebuild when new snapshots arrive, so lists and detail views stay in sync with the database without manual refresh.

Key data-flow scenarios

- **Login and profile** — User enters credentials → Firebase Auth sign-in → on success, app calls `ProfileService.initializeUserProfile()` which may read then create or skip creating a `staff_profiles` document; profile screen reads profile via `getUserProfile(uid)` (Firestore get) and displays it. Edit profile: form → validation → optional Storage upload for new image → Firestore update on `staff_profiles.doc(uid)` → success feedback.

- **Report incident** — Form (category, description, location, optional photo) → validation → optional `putFile` to Storage under `incidents/{userId}/...` → `getDownloadURL` → Firestore `staff_incidents.add({ ... photoUrl })` with server timestamps → UI success and navigation back.
- **View incidents / alerts** — Screen subscribes to `staff_incidents` or `alerts` (with filters such as `status == 'active'` or `reportedBy == uid`). Firestore stream pushes documents → app maps to list items → `StreamBuilder` rebuilds the list when data changes.
- **Admin update incident status** — Admin selects new status and confirms → app calls `staff_incidents.doc(id).update({ status, updatedAt })` → Firestore updates → listeners on admin and staff sides receive the new snapshot and UI updates accordingly.
- **Delete incident** — Staff confirms delete → `staff_incidents.doc(id).delete()` → document removed → listeners receive update and list removes the item (or user is navigated back).

Consistency and integrity

- **Validation** — Required fields and formats are checked in the app before any write, reducing invalid or incomplete documents.
- **Server timestamps** — The app uses `FieldValue.serverTimestamp()` for `createdAt` and `updatedAt`, so times are set by Firestore and are consistent across clients.
- **Security rules** — Firestore and Storage rules enforce who can read/write what (e.g. profile write only when `request.auth.uid == userId`), so the storage layer rejects unauthorized or inconsistent access even if the client is tampered with.
- **Real-time sync** — Using Firestore listeners keeps the UI aligned with the database; if data is changed from another device or by another user, the app receives the update and re-renders, avoiding stale views.
- **Denormalised fields** — Storing `reportedByName`, `reportedByEmail`, and `createdByName` in incidents and alerts avoids extra lookups and keeps list views consistent even if a user later changes their profile.

The following diagram summarises the high-level data flow between UI, application logic, and the storage layer.

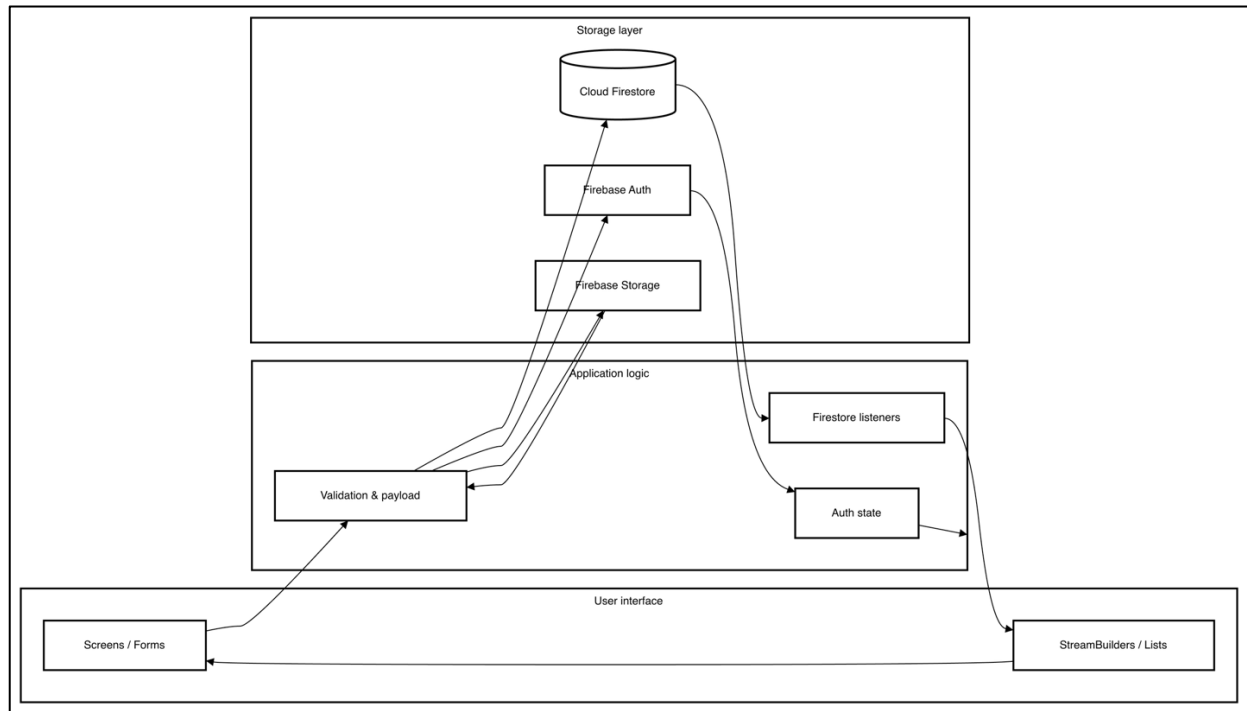


Figure 4.3: Data flow between UI, application logic, and storage. User input goes through validation and then to Firebase; Firestore and Auth push data back via listeners and auth state to update the UI.

5. CRUD Operations

The system consists of CRUD operations to make the app functional and aligned with the requirement. Below are the detailed operations for each app modules.

5.1 Create Operations

The system supports data creation for key functionalities such as user registration, incident reporting, and alert broadcasting.

All operations that create new data (Firestore documents or Storage files).

Table 5.1 Shows the create operation.

Resource	Location in app	Justification
Staff profile	<code>sign_up.dart</code> — <code>staff_profiles.doc(uid).set(...)</code> when registering; <code>profile_service.dart</code> — <code>initializeUserProfile()</code> uses <code>.set()</code> if profile does not exist	New users need a Firestore profile (name, email, department, etc.) for the staff directory and profile screens.
Staff incident	<code>staff_report_incident.dart</code> — <code>staff_incidents.add({category, description, location, reportedBy, status, photoUrl, ... })</code>	Staff report safety/equipment/security issues; each report is one new incident document.
Alert	<code>admin_sendalertscreen.dart</code> — <code>alerts.add({ alertType, title, description, sendTo, imageUrl, createdBy, ... })</code>	Admins broadcast alerts to staff; each sent alert is one new document.
Incident photo (Storage)	<code>staff_report_incident.dart</code> — <code>FirebaseStorage.ref().putFile()</code> then <code>getDownloadURL()</code> ; URL stored in <code>staff_incidents.photoUrl</code>	Optional photo evidence for incident reports.
Profile image (Storage)	<code>staff_profile_edit.dart</code> — upload to <code>profiles/{userId}/...</code> , URL saved in <code>staff_profiles.profileImageUrl</code>	Staff can set or change their profile picture.

5.2 Read Operations

Users and administrators can retrieve relevant data including alert history, incident status, and response acknowledgments.

All operations that read data (Firestore documents or Storage files).

Table 5.2 Shows the read operation.

Resource	Location in app	Justification
Staff profiles	<code>profile_service.dart</code> — <code>getUserProfile(uid)</code> (single doc <code>.get()</code>); <code>admin_staff_directory.dart</code> — <code>staff_profiles.snapshots()</code> ; sign-up — <code>.get()</code> to check if profile exists	Load profile for profile view/edit, staff directory list, and sign-up logic.
Staff incidents	<code>admin_dashboard.dart</code> , <code>admin_all_incidents.dart</code> , <code>admin_view_incident.dart</code> — streams / doc reads; <code>staff_dashboard.dart</code> , <code>incidents.dart</code> , <code>staff_incident_details.dart</code> — streams / doc reads	Admins list and view all incidents; staff list and view their incidents.
Alerts	<code>admin_dashboard.dart</code> , <code>staff_dashboard.dart</code> , <code>staff_alert_history.dart</code> — <code>alerts.snapshots()</code>	Admins and staff see current and past alerts.
Incident / profile images	Any screen that displays <code>photoUrl</code> or <code>profileImageUrl</code> (e.g. incident details, profile, directory)	Show uploaded photos; Storage read is implicit via the stored URL.

5.3 Update Operations

The application allows updates to records such as incident status changes, user profile modifications, and response confirmations.

All operations that update existing data (no new document; only field updates).

Table 5.3 Shows the update operation.

Resource	Location in app	Justification
Staff profile	<code>staff_profile_edit.dart</code> — <code>staff_profiles.doc(uid).update({ fullName, phone, department, profileImageUrl, updatedAt })</code> ; <code>sign_up.dart</code> — <code>.update()</code> when profile already exists (e.g. name, phone)	Staff edit their own profile; sign-up can refresh profile if user already had one.
Staff incident	<code>admin_view_incident.dart</code> — <code>staff_incidents.doc(id).update({ status, updatedAt })</code>	Admins change incident status (e.g. pending → in progress → resolved).

Alerts	—	Not implemented; alerts are treated as immutable once sent.
Staff profile (Firestore rules)	—	

5.4 Delete Operations

All operations that delete data (Firestore documents; Storage delete is not used in the app). Controlled deletion mechanisms are implemented for managing outdated or invalid data while maintaining data integrity and audit requirements.

Table 5.4 Shows the delete operation.

Resource	Location in app	Justification
Staff incident	<code>staff_incident_details.dart — staff_incidents.doc(documentId).delete()</code> (with confirmation dialog)	Staff can remove their own report (e.g. duplicate or mistaken submission).
Staff profile	Not implemented	Firestore rules disallow profile delete (<code>allow delete: if false</code>) to preserve audit trail and directory consistency.
Alert	Not implemented	Alerts remain as history; no retract/delete feature.
Storage files	Not implemented	No UI or backend logic to delete incident or profile images.

6. Important Source Code Snippets

This section highlights selected source code snippets that represent critical system functionalities: authentication and role handling, alert broadcasting, incident reporting, and data retrieval and update operations. The snippets illustrate coding practices and technical implementation rather than full program listings.

6.1 Authentication and Role Handling

After the user signs in with email and password, the app ensures a Firestore profile exists and then routes the user by role (staff or admin). Role is taken from the `staff_profiles` document field `userType`, with a fallback for new users based on email in `ProfileService`.

Sign-in and role-based navigation (`lib/login/login.dart`):

```
// Firebase Authentication Sign-In
final userCredential = await FirebaseAuth.instance.signInWithEmailAndPassword(
  email: email,
  password: password,
);

if (userCredential.user != null) {
  await ProfileService.initializeUserProfile(userCredential.user!);

  final profileData = await
ProfileService.getUserProfile(userCredential.user!.uid);
  final userType = profileData?['userType'] ?? 'staff';
  final isStaff = userType == 'staff';

  // Navigate to the appropriate screen based on user role
  Navigator.pushReplacement(
    context,
    MaterialPageRoute(
      builder: (context) => isStaff ? StaffDashboard() : AdminDashboard(),
    ),
  );
}
```

User type determination for new profiles (`lib/services/profile_service.dart`):

```
String userType = 'staff';
final email = user.email?.toLowerCase() ?? '';
if (email.contains('admin') || email == 'admin@workbeacon.com') {
  userType = 'admin';
}
// ... then stored in staff_profiles document
```

Together, these snippets show: Firebase Auth for sign-in, a dedicated service for profile init and fetch, and routing driven by Firestore `userType` (with an email-based default for new profiles).

6.2 Alert Broadcasting Logic

Admins send alerts by building a document map and adding it to the `alerts` collection. Optional image upload is done before building the payload; the resulting URL is stored in the alert document.

Building and saving an alert (`lib/screens/admin/admin_sendalerts.dart`):

```
Map<String, dynamic> alertData = {
  'alertType': _selectedAlertType,
  'title': _titleController.text.trim(),
  'description': _descriptionController.text.trim(),
  'location': _locationController.text.trim().isEmpty ? null :
    _locationController.text.trim(),
  'sendTo': _selectedSendTo == 'all' ? 'all' : _departmentController.text.trim(),
  'requireAcknowledgment': _requireAcknowledgment,
  'imageUrl': imageUrl,
  'createdBy': user.uid,
  'createdByEmail': user.email,
  'createdByName': user.displayName ?? 'Admin',
  'createdAt': FieldValue.serverTimestamp(),
  'status': 'active',
};

await FirebaseFirestore.instance.collection('alerts').add(alertData);
```

This demonstrates: validation and payload construction in the UI layer, use of server timestamps and denormalised creator fields, and a single Firestore add for broadcasting.

6.3 Incident Reporting Mechanism

Staff submit incidents by optionally uploading a photo to Storage, then creating a `staff_incidents` document that includes the photo URL. The snippet shows the upload helper and the Firestore write.

Image upload for incident (`lib/screens/staff/staff_report_incident.dart`):

```
final String fileName =

'incidents/${user.uid}/${DateTime.now().millisecondsSinceEpoch}_${_selectedImage!.name}';

final Reference storageRef = FirebaseStorage.instance.ref().child(fileName);
final UploadTask uploadTask = storageRef.putFile(File(_selectedImage!.path));
final TaskSnapshot snapshot = await uploadTask;
final String downloadUrl = await snapshot.ref.getDownloadUrl();
return downloadUrl;
```

Adding the incident document (same file):

```
await FirebaseFirestore.instance.collection('staff_incidents').add({
  'category': _selectedCategory,
  'description': _descriptionController.text.trim(),
  'location': _locationController.text.trim().isEmpty ? null :
    _locationController.text.trim(),
  'reportedBy': user.uid,
  'reportedByEmail': user.email,
```

```

    'reportedByName': user.displayName ?? 'Unknown',
    'status': 'pending',
    'photoUrl': imageUrl,
    'createdAt': FieldValue.serverTimestamp(),
    'updatedAt': FieldValue.serverTimestamp(),
  });

```

Together, these show: Storage path convention (incidents/{userId}/...), async upload then URL use in Firestore, and consistent use of server timestamps and reporter fields.

6.4 Data Retrieval and Update Operations

Real-time retrieval with StreamBuilder — Staff incident list uses a Firestore stream filtered by reportedBy so each user sees only their reports and the UI updates when data changes (lib/screens/staff/incidents.dart):

```

StreamBuilder<QuerySnapshot>(
  stream: FirebaseFirestore.instance
    .collection('staff_incidents')
    .where('reportedBy', isEqualTo: user.uid)
    .snapshots(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return Center(child: CircularProgressIndicator());
    }
    if (snapshot.hasError) { /* error UI */ }
    // ... build list from snapshot.data?.docs
  },
)

```

One-time profile read (lib/services/profile_service.dart):

```

static Future<Map<String, dynamic?>> getUserProfile(String uid) async {
  final profileDoc = await _firestore.collection('staff_profiles').doc(uid).get();
  if (profileDoc.exists) {
    return profileDoc.data();
  }
  return null;
}

```

Update operations — Admin updates incident status (lib/screens/admin/admin_view_incident.dart):

```

await FirebaseFirestore.instance
  .collection('staff_incidents')
  .doc(widget.documentId)
  .update({
    'status': statusToSave,
    'updatedAt': FieldValue.serverTimestamp(),
  });

```

Staff profile update (lib/screens/staff/staff_profile_edit.dart):

```

await FirebaseFirestore.instance
  .collection('staff_profiles')
  .doc(user.uid)
  .update({
    'fullName': _fullNameController.text.trim(),
  });

```

```
'phone': _phoneController.text.trim(),  
'department': _selectedDepartment,  
if (imageUrl != null) 'profileImageUrl': imageUrl,  
'updatedAt': FieldValue.serverTimestamp(),  
});
```

These snippets illustrate: real-time lists via `snapshot()` and `StreamBuilder`, one-off reads with `.get()`, and targeted `.update()` with server timestamp and conditional fields (e.g. `profileImageUrl` only when a new image is uploaded).

7. Technical Challenges and Solutions

7.1 Identified Challenges

During the development of WorkBeacon, the team identified the following technical and design challenges.

1. Firestore security rules and permissions

Getting Firestore security rules right so that authenticated users could only access the intended data (e.g. staff seeing only their own incidents, admins seeing all) caused permission errors during development. Queries sometimes failed with “Missing or insufficient permissions” or similar, especially when filters (e.g. `where('reportedBy', isEqualTo: uid)` or `where('status', isEqualTo: 'active')`) were used. The app was updated to detect permission and index-related errors and show clear messages (e.g. asking users to check Firestore security rules or create the suggested composite index) instead of generic failures. Balancing security with the need for different read/write access for staff and admins required several iterations on the rules.

2. Profile initialization and role assignment

Ensuring every signed-in user had a corresponding `staff_profiles` document, and that role (`userType`) was set correctly for routing to staff or admin dashboards, was tricky. Existing users created before the profile system might have no document; new sign-ups create the profile at registration, while login uses `ProfileService.initializeUserProfile()` to create one if missing. Determining admin vs staff when creating a new profile relied on a fallback (e.g. email containing "admin" or a known admin address). The team chose to catch profile errors and allow login or sign-up to proceed so that authentication was not blocked by a single Firestore write failure.

3. Image upload and storage reliability

Incident photos and profile/alert images depend on Firebase Storage uploads and then storing the download URL in Firestore. Challenges included handling pick/upload failures (e.g. user cancels, network errors, or large files), showing clear error messages, and handling missing or invalid URLs when displaying images (e.g. using `errorBuilder` on image widgets so broken links do not break the UI). The team had to ensure the sequence “pick image → upload → get URL → write Firestore” was robust and that the UI stayed consistent when upload or write failed.

4. Real-time location and live map

The admin “live locations” feature was intended to show staff on a map. Implementing real-time GPS or device location, persisting it (e.g. in Firestore), and updating the map for all admins would have required background location, privacy handling, and real-time listeners. Given scope and time, the team implemented a map UI with **dummy/simulated** staff positions (fixed list of names and random positions near a centre point) so that the map and filtering UX could be demonstrated without full location infrastructure. Delivering real-time staff locations end-to-end was therefore deferred as a known limitation.

5. Optional configuration and environment

The app supports optional configuration (e.g. Mapbox token in `.env` for custom map tiles). Loading `.env` must not prevent the app from starting if the file is missing or invalid. The team used a try-catch around `dotenv.load()` so that startup always continues and the app falls back to default map tiles when no token is set. Handling optional env and documenting it (e.g. in README and `.env.example`) required clear separation between required and optional config.

6. Composite indexes for Firestore queries

Several screens use filtered or ordered queries (e.g. alerts by `status`, incidents by `reportedBy` or by `createdAt`). Firestore often requires **composite indexes** for such queries. When indexes were missing, the app received runtime errors. The team had to create the corresponding indexes (e.g. via Firebase Console or `firestore.indexes.json`) and added error handling that detects index-related errors and shows a user-friendly message so that users and developers could identify the need to create or fix an index.

7. Cross-platform setup and testing

Supporting both Android and iOS (and optionally web/desktop) meant managing two Firebase config files (`google-services.json` and `GoogleService-Info.plist`), platform-specific build settings, and testing on emulators and real devices. Ensuring consistent behaviour across platforms (e.g. image picker, Firebase init, and navigation) required coordinated setup and testing on each target.

8. User experience during loading and errors

Where data is loaded from Firestore (e.g. streams) or from Storage, the UI had to handle loading, empty, and error states consistently. The team used patterns such as `StreamBuilder` with checks for `connectionState`, `hasError`, and `empty docs`, and provided loading indicators, empty-state messages, and specific error messages (e.g. for permission or index issues) so that users were not left with blank screens or unclear failures.

7.2 Implemented Solutions

Each identified challenge was addressed through appropriate technical solutions, design adjustments, or framework features to ensure system stability, usability, and performance.

1. Firestore security rules and permissions

The team defined Firestore rules so that access is tied to authentication and, where needed, to the document owner. For `staff_profiles`, `read` is allowed for any authenticated user, while `create` and `update` are allowed only when `request.auth.uid == userId`; `delete` is disabled. For `staff_incidents` and `alerts`, `read` and `write` are allowed for any authenticated user, with role-based behaviour (`staff` vs `admin`) enforced in the application logic. Where queries required composite indexes, the team created them via the Firebase Console (or `firestore.indexes.json`) and added error handling that detects permission- and index-related errors and shows clear messages (e.g. “Permission denied. Please check your Firestore security rules” or guidance to add the suggested index), so that failures are understandable and fixable without breaking the app.

2. Profile initialization and role assignment

A dedicated **ProfileService** was introduced to centralize profile creation and lookup. On login, the app calls `ProfileService.initializeUserProfile(user)`, which checks for an existing `staff_profiles` document; if none exists, it creates one with defaults and sets `userType` using a fallback (e.g. "admin" when the email contains "admin" or matches a known admin address, otherwise "staff"). Role-based routing then uses `ProfileService.getUserProfile(uid)` to read `userType` and navigate to `StaffDashboard` or `AdminDashboard`. Profile initialization and fetch are wrapped in try-catch so that Firestore errors are logged and do not block login or sign-up, preserving system stability and usability even when the profile write or read fails.

3. Image upload and storage reliability

The flow was standardized to **upload first, then write the document**: the app uploads the file to Firebase

Storage (e.g. `incidents/{userId}/{timestamp}_{filename}` or `profiles/{userId}/...`), awaits the upload and retrieves the download URL, then writes the Firestore document with that URL (or `null` if no image or upload failed). Image pick and upload are wrapped in try-catch with user-visible `SnackBar` messages so that pick cancellation, network errors, or upload failures do not crash the app. When displaying images, the UI uses the `errorBuilder` parameter of image widgets to show a placeholder or icon when the URL is missing or invalid, ensuring usability and avoiding blank or broken image areas.

4. Real-time location and live map

Given scope and time, the team chose a **design adjustment** rather than full GPS and real-time persistence: the admin live locations screen uses a fixed list of dummy staff (names and roles) with positions generated randomly near a fixed map centre. The map UI, markers, and filtering are fully implemented so that the feature is demonstrable and the UX is consistent; real-time device location and Firestore-backed positions were documented as future work. This kept the system stable and deliverable while making the limitation explicit.

5. Optional configuration and environment

Environment loading was made **non-blocking**: in `main()`, `dotenv.load(fileName: '.env')` is called inside a try-catch so that a missing or invalid `.env` file does not prevent app startup. When the Mapbox token is absent, the app continues and uses default or open map tiles. An `.env.example` file and README instructions describe optional variables (e.g. `MAPBOX_ACCESS_TOKEN`), so that configuration is clear and the app remains stable across different developer and deployment environments.

6. Composite indexes for Firestore queries

For each query that required a composite index (e.g. filtering by `status` and ordering by `createdAt`), the team created the corresponding index in the Firebase Console (or via deployment config). In addition, the app checks for index-related errors in Firestore stream/query error handlers and shows a user-friendly message (e.g. that an index may be required and that the error details contain a link to create it). This improved usability during development and for anyone deploying the app with new queries, and avoided silent or cryptic failures.

7. Cross-platform setup and testing

The team used **platform-specific Firebase configuration** (e.g. `google-services.json` for Android and `GoogleService-Info.plist` for iOS) and

documented in the README where to place these files and how to enable Auth, Firestore, and Storage. Build and run instructions for Android and iOS (and optional targets) were specified so that all targets could be built and tested. This ensured consistent behaviour across platforms and reduced setup errors.

8. User experience during loading and errors

Where data is loaded from Firestore, the UI uses **StreamBuilder** (or equivalent) and explicitly handles three states: **loading** (`connectionState == ConnectionState.waiting` → e.g. `CircularProgressIndicator`), **error** (`snapshot.hasError` → message and optional retry or guidance), and **data** (empty list vs list with items, with appropriate empty-state text). Permission and index errors are parsed from the error message and replaced with clearer text. This design keeps the system stable under failure and preserves usability by avoiding blank screens and generic error messages.

8. Conclusion

WorkBeacon is a mobile application for workplace safety, incident reporting, and staff visibility, developed by the MemeMaker team for SECJ3623 Pengaturcaraan Aplikasi Mobil. This report has documented its development approach, technical design, and implementation.

The project followed an iterative, feature-driven methodology and a clear workflow from requirement analysis and UI/UX design through development, testing, and deployment preparation. Task tracking and progress were supported by GitHub Project and the project repository, helping the team deliver on time and maintain quality.

The development environment was built around the Flutter SDK and Dart, with Firebase (Authentication, Cloud Firestore, and Storage) as the backend. Supporting tools for version control, project management, UI design, and testing improved collaboration and code quality. The data storage architecture is cloud-centric: the Flutter app communicates with Firebase only, with no custom server. Firestore holds staff profiles, incidents, and alerts; Firebase Storage holds incident and profile images; and security is enforced by Firestore and Storage rules. Data models and data flow between the UI, application logic, and storage were described, and all CRUD operations were listed and justified by operation type (Create, Read, Update, Delete). Selected source code snippets illustrated authentication and role handling, alert broadcasting, incident reporting, and data retrieval and update operations. The report also outlined technical challenges—such as Firestore rules and permissions, profile and role handling, image upload reliability, and real-time location scope—and the solutions adopted to maintain stability, usability, and performance.

In summary, WorkBeacon meets its intended goals: staff can report incidents with optional photos and view alerts; administrators can manage incidents, send alerts, browse the staff directory, and use a map-based view (currently with simulated locations). The combination of Flutter, Firebase, and the described design choices resulted in a working application that can be extended with real-time staff location, stricter role-based rules, or additional safety features in future work.